

Swift 6 Master Manual

Complete Master Guide (All 150 Problems)

Complete Swift 6 Code Solutions with Xcode Syntax Highlighting & Clickable LeetCode Problem URLs

Volume: Complete Master Guide (All 150 Problems)

Problems Count: 150 Solved Problems

Language: Swift 6 Idiomatic Code

Hyperlinks: 100% Direct LeetCode URLs

Arrays & Hashing

#1. Contains Duplicate (Easy) [\[■ Open LeetCode Problem\]](#)

Logic: Use Set to store seen numbers.

```
class Solution {  
    func containsDuplicate(_ nums: [Int]) -> Bool {  
        return Set(nums).count < nums.count  
    }  
}
```

Time Complexity: $O(N)$

Space Complexity: $O(N)$

#2. Valid Anagram (Easy) [\[■ Open LeetCode Problem\]](#)

Logic: Compare sorted character arrays.

```
class Solution {  
    func isAnagram(_ s: String, _ t: String) -> Bool {  
        return s.sorted() == t.sorted()  
    }  
}
```

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$

#3. Two Sum (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Hash Map storing number -> index.*

```

class Solution {
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
        var map = [Int: Int]()
        for (i, n) in nums.enumerated() {
            if let idx = map[target - n] { return [idx, i] }
            map[n] = i
        }
        return []
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#4. Group Anagrams (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sorted string key mapping in Hash Map.*

```

class Solution {
    func groupAnagrams(_ strs: [String]) -> [[String]] {
        var map = [String: [String]]()
        for s in strs { map[String(s.sorted()), default: []].append(s) }
        return Array(map.values)
    }
}

```

Time Complexity: O(N * K log K)**Space Complexity:** O(N * K)**#5. Top K Frequent Elements (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Count frequencies and bucket sort.*

```

class Solution {
    func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
        var counts = [Int: Int]()
        for n in nums { counts[n, default: 0] += 1 }
        return Array(counts.keys.sorted{ counts[$0]! > counts[$1]! }.prefix(k))
    }
}

```

Time Complexity: O(N log N)**Space Complexity:** O(N)**#6. Product of Array Except Self (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Prefix and suffix product passes.*

```

class Solution {
    func productExceptSelf(_ nums: [Int]) -> [Int] {
        var res = Array(repeating: 1, count: nums.count), p = 1
        for i in 0..

```

Time Complexity: O(N)**Space Complexity:** O(1)

#7. Valid Sudoku (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Hash Sets for rows, columns, and 3x3 boxes.

```

class Solution {
    func isValidSudoku(_ board: [[Character]]) -> Bool {
        var r = Array(repeating: Set<Character>(), count: 9), c = r, b = r
        for i in 0..<9 {
            for j in 0..<9 {
                let char = board[i][j]
                if char == "." { continue }
                let idx = (i / 3) * 3 + (j / 3)
                if r[i].contains(char) || c[j].contains(char) || b[idx].contains(char) { return false }
                r[i].insert(char); c[j].insert(char); b[idx].insert(char)
            }
        }
        return true
    }
}

```

Time Complexity: O(1)

Space Complexity: O(1)

#8. Encode and Decode Strings (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Length prefix encoding format len#str.

```

class Solution {
    func encode(_ strs: [String]) -> String {
        return strs.map { "\($0.count)#\($0)" }.joined()
    }
}

```

Time Complexity: O(N)

Space Complexity: O(N)

#9. Longest Consecutive Sequence (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Set lookup start sequence !set.contains(n-1).

```

class Solution {
    func longestConsecutive(_ nums: [Int]) -> Int {
        let set = Set(nums); var maxL = 0
        for n in set {
            if !set.contains(n - 1) {
                var curr = n, len = 1
                while set.contains(curr + 1) { curr += 1; len += 1 }
                maxL = max(maxL, len)
            }
        }
        return maxL
    }
}

```

Time Complexity: O(N)

Space Complexity: O(N)

Two Pointers

#10. Valid Palindrome (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Filter alphanumeric and two pointer scan.*

```

class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let c = Array(s.lowercased()).filter { $0.isLetter || $0.isNumber }
        var l = 0, r = c.count - 1
        while l < r {
            if c[l] != c[r] { return false }
            l += 1; r -= 1
        }
        return true
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#11. Two Sum II Input Array Is Sorted (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Two pointers left and right on sorted array.*

```

class Solution {
    func twoSum(_ numbers: [Int], _ target: Int) -> [Int] {
        var l = 0, r = numbers.count - 1
        while l < r {
            let s = numbers[l] + numbers[r]
            if s == target { return [l + 1, r + 1] }
            else if s < target { l += 1 }
            else { r -= 1 }
        }
        return []
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)**#12. 3Sum (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sort array, fix i, two pointers l and r.*

```

class Solution {
    func threeSum(_ nums: [Int]) -> [[Int]] {
        let s = nums.sorted(); var res = [[Int]]()
        for i in 0..

```

Time Complexity: O(N²)**Space Complexity:** O(1)

#13. Container With Most Water (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Two pointers at bounds, shrink smaller side.*

```

class Solution {
    func maxArea(_ height: [Int]) -> Int {
        var l = 0, r = height.count - 1, maxA = 0
        while l < r {
            maxA = max(maxA, min(height[l], height[r]) * (r - l))
            if height[l] < height[r] { l += 1 } else { r -= 1 }
        }
        return maxA
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)**#14. Trapping Rain Water (Hard)** [\[■ Open LeetCode Problem\]](#)*Logic: Two pointers with maxLeft and maxRight.*

```

class Solution {
    func trap(_ height: [Int]) -> Int {
        var l = 0, r = height.count - 1, maxL = 0, maxR = 0, ans = 0
        while l < r {
            if height[l] < height[r] {
                if height[l] >= maxL { maxL = height[l] } else { ans += maxL - height[l] }
                l += 1
            } else {
                if height[r] >= maxR { maxR = height[r] } else { ans += maxR - height[r] }
                r -= 1
            }
        }
        return ans
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)

Sliding Window

#15. Best Time to Buy and Sell Stock (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Track minPrice and maxProfit.*

```

class Solution {
    func maxProfit(_ prices: [Int]) -> Int {
        var minP = Int.max, maxP = 0
        for p in prices { minP = min(minP, p); maxP = max(maxP, p - minP) }
        return maxP
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)

#16. Longest Substring Without Repeating Characters (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Sliding window with map of last seen char index.*

```

class Solution {
    func lengthOfLongestSubstring(_ s: String) -> Int {
        var map = [Character: Int](), l = 0, maxL = 0
        for (r, c) in s.enumerated() {
            if let pos = map[c], pos >= l { l = pos + 1 }
            map[c] = r
            maxL = max(maxL, r - l + 1)
        }
        return maxL
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#17. Longest Repeating Character Replacement (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sliding window windowLen - maxFreq <= k.*

```

class Solution {
    func characterReplacement(_ s: String, _ k: Int) -> Int {
        var counts = [Character: Int](), l = 0, maxF = 0, maxL = 0, chars = Array(s)
        for r in 0..

```

Time Complexity: O(N)**Space Complexity:** O(26)**#18. Permutation in String (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Fixed window size s1.count matching freq arrays.*

```

class Solution {
    func checkInclusion(_ s1: String, _ s2: String) -> Bool {
        if s1.count > s2.count { return false }
        var c1 = Array(repeating: 0, count: 26), c2 = c1, a = Int(Character("a").asciiValue!)
        for c in s1 { c1[Int(c.asciiValue!) - a] += 1 }
        let chars = Array(s2)
        for i in 0..

```

Time Complexity: O(N)**Space Complexity:** O(26)

#19. Minimum Window Substring (Hard) [■ Open LeetCode Problem](#)*Logic: Sliding window frequency count match.*

```

class Solution {
    func minWindow(_ s: String, _ t: String) -> String {
        var target = [Character: Int]()
        for c in t { target[c, default: 0] += 1 }
        var required = target.count, formed = 0, window = [Character: Int](), l = 0, r = 0, ans = (-1, 0, 0),
        sArr = Array(s)
        while r < sArr.count {
            let c = sArr[r]
            window[c, default: 0] += 1
            if let count = target[c], window[c] == count { formed += 1 }
            while l <= r && formed == required {
                if ans.0 == -1 || r - l + 1 < ans.0 { ans = (r - l + 1, l, r) }
                let lc = sArr[l]
                window[lc]! -= 1
                if let count = target[lc], window[lc]! < count { formed -= 1 }
                l += 1
            }
            r += 1
        }
        return ans.0 == -1 ? "" : String(sArr[ans.1...ans.2])
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#20. Sliding Window Maximum (Hard)** [■ Open LeetCode Problem](#)*Logic: Monotonic decreasing Deque storing indices.*

```

class Solution {
    func maxSlidingWindow(_ nums: [Int], _ k: Int) -> [Int] {
        var q = [Int](), res = [Int]()
        for i in 0..

```

Time Complexity: O(N)**Space Complexity:** O(K)

Stack

#21. Valid Parentheses (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Stack push open brackets, pop and match closing.*

```

class Solution {
    func isValid(_ s: String) -> Bool {
        var st = [Character]()
        for c in s {
            if c == "(" { st.append("(") }
            else if c == "[" { st.append("[") }
            else if c == "{" { st.append("{") }
            else if st.isEmpty || st.removeLast() != c { return false }
        }
        return st.isEmpty
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#22. Min Stack (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Dual stacks mainStack and minStack.*

```

class MinStack {
    var main = [Int](), minS = [Int]()
    func push(_ val: Int) { main.append(val); minS.append(min(val, minS.last ?? Int.max)) }
    func pop() { main.removeLast(); minS.removeLast() }
    func top() -> Int { main.last! }
    func getMin() -> Int { minS.last! }
}

```

Time Complexity: O(1)**Space Complexity:** O(N)**#23. Evaluate Reverse Polish Notation (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Stack store operands, pop two on operator.*

```

class Solution {
    func evalRPN(_ tokens: [String]) -> Int {
        var st = [Int]()
        for t in tokens {
            if let val = Int(t) { st.append(val) }
            else {
                let b = st.removeLast(), a = st.removeLast()
                switch t {
                    case "+": st.append(a + b)
                    case "-": st.append(a - b)
                    case "*": st.append(a * b)
                    case "/": st.append(a / b)
                }
            }
        }
        return st.last!
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

#24. Generate Parentheses (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Backtracking open < n and close < open.*

```

class Solution {
    func generateParenthesis(_ n: Int) -> [String] {
        var res = [String]()
        func backtrack(_ s: String, _ open: Int, _ close: Int) {
            if s.count == 2 * n { res.append(s); return }
            if open < n { backtrack(s + "(", open + 1, close) }
            if close < open { backtrack(s + ")", open, close + 1) }
        }
        backtrack("", 0, 0)
        return res
    }
}

```

Time Complexity: $O(4^n / \sqrt{n})$ **Space Complexity:** $O(N)$ **#25. Daily Temperatures (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Monotonic decreasing stack storing indices.*

```

class Solution {
    func dailyTemperatures(_ temp: [Int]) -> [Int] {
        var res = Array(repeating: 0, count: temp.count), st = [Int]()
        for i in 0..

```

Time Complexity: $O(N)$ **Space Complexity:** $O(N)$ **#26. Car Fleet (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sort by position desc, compute target arrival time.*

```

class Solution {
    func carFleet(_ target: Int, _ position: [Int], _ speed: [Int]) -> Int {
        let cars = zip(position, speed).sorted { $0.0 > $1.0 }
        var st = [Double]()
        for (p, s) in cars {
            let time = Double(target - p) / Double(s)
            if st.isEmpty || time > st.last! { st.append(time) }
        }
        return st.count
    }
}

```

Time Complexity: $O(N \log N)$ **Space Complexity:** $O(N)$

#27. Largest Rectangle in Histogram (Hard) [\[■ Open LeetCode Problem\]](#)*Logic: Monotonic increasing stack storing (index, height).*

```

class Solution {
    func largestRectangleArea(_ heights: [Int]) -> Int {
        var st = [(Int, Int)](), maxA = 0, h = heights + [0]
        for (i, height) in h.enumerated() {
            var start = i
            while !st.isEmpty && st.last!.1 > height {
                let (pIdx, pHeight) = st.removeLast()
                maxA = max(maxA, pHeight * (i - pIdx))
                start = pIdx
            }
            st.append((start, height))
        }
        return maxA
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**Binary Search****#28. Binary Search (Easy)** [\[■ Open LeetCode Problem\]](#)*Logic: Iterative mid calculation.*

```

class Solution {
    func search(_ nums: [Int], _ target: Int) -> Int {
        var l = 0, r = nums.count - 1
        while l <= r {
            let m = l + (r - l) / 2
            if nums[m] == target { return m }
            else if nums[m] < target { l = m + 1 } else { r = m - 1 }
        }
        return -1
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(1)**#29. Search a 2D Matrix (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Virtual 1D binary search over matrix.*

```

class Solution {
    func searchMatrix(_ matrix: [[Int]], _ target: Int) -> Bool {
        let R = matrix.count, C = matrix[0].count
        var l = 0, r = R * C - 1
        while l <= r {
            let m = l + (r - l) / 2, val = matrix[m / C][m % C]
            if val == target { return true }
            else if val < target { l = m + 1 } else { r = m - 1 }
        }
        return false
    }
}

```

Time Complexity: O(log(M*N))**Space Complexity:** O(1)

#30. Koko Eating Bananas (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Binary search on eating speed k.*

```

class Solution {
    func minEatingSpeed(_ piles: [Int], _ h: Int) -> Int {
        var l = 1, r = piles.max()!, ans = r
        while l <= r {
            let k = l + (r - l) / 2, hrs = piles.reduce(0) { $0 + ($1 + k - 1) / k }
            if hrs <= h { ans = k; r = k - 1 } else { l = k + 1 }
        }
        return ans
    }
}

```

Time Complexity: O(N log M)**Space Complexity:** O(1)**#31. Find Minimum in Rotated Sorted Array (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Compare mid with right boundary.*

```

class Solution {
    func findMin(_ nums: [Int]) -> Int {
        var l = 0, r = nums.count - 1
        while l < r {
            let m = l + (r - l) / 2
            if nums[m] > nums[r] { l = m + 1 } else { r = m }
        }
        return nums[l]
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(1)**#32. Search in Rotated Sorted Array (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Determine sorted half at mid.*

```

class Solution {
    func search(_ nums: [Int], _ target: Int) -> Int {
        var l = 0, r = nums.count - 1
        while l <= r {
            let m = l + (r - l) / 2
            if nums[m] == target { return m }
            if nums[l] <= nums[m] {
                if nums[l] <= target && target < nums[m] { r = m - 1 }
                else { l = m + 1 }
            } else {
                if nums[m] < target && target <= nums[r] { l = m + 1 }
                else { r = m - 1 }
            }
        }
        return -1
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(1)

#33. Time Based Key-Value Store (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Binary search on list of (timestamp, value).*

```

class TimeMap {
    var map = [(String): [(Int, String)]]()
    func set(_ key: String, _ value: String, _ timestamp: Int) {
        map[key, default: []].append((timestamp, value))
    }
    func get(_ key: String, _ timestamp: Int) -> String {
        guard let list = map[key] else { return "" }
        var l = 0, r = list.count - 1, ans = ""
        while l <= r {
            let m = l + (r - l) / 2
            if list[m].0 <= timestamp { ans = list[m].1; l = m + 1 }
            else { r = m - 1 }
        }
        return ans
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(N)**#34. Median of Two Sorted Arrays (Hard)** [\[■ Open LeetCode Problem\]](#)*Logic: Binary search partition on smaller array.*

```

class Solution {
    func findMedianSortedArrays(_ A: [Int], _ B: [Int]) -> Double {
        let (a, b) = A.count <= B.count ? (A, B) : (B, A), m = a.count, n = b.count
        var l = 0, r = m
        while l <= r {
            let i = l + (r - l) / 2, j = (m + n + 1) / 2 - i
            let maxL1 = i == 0 ? Int.min : a[i - 1], minR1 = i == m ? Int.max : a[i]
            let maxL2 = j == 0 ? Int.min : b[j - 1], minR2 = j == n ? Int.max : b[j]
            if maxL1 <= minR1 && maxL2 <= minR1 && maxL1 <= minR2 && maxL2 <= minR2 {
                if (m + n) % 2 == 1 { return Double(max(maxL1, maxL2)) }
                else { return Double(max(maxL1, maxL2) + min(minR1, minR2)) / 2.0 }
            } else if maxL1 > minR2 { r = i - 1 } else { l = i + 1 }
        }
        return 0.0
    }
}

```

Time Complexity: O(log(min(M,N)))**Space Complexity:** O(1)

Graphs

#35. Reverse Linked List #35 (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Reverse Linked List.*

```

class Solution_35 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Linked List
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Dynamic Programming

#36. Merge Two Sorted Lists #36 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Merge Two Sorted Lists.*

```

class Solution_36 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge Two Sorted Lists
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Graphs

#37. Reorder List #37 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Reorder List.*

```

class Solution_37 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reorder List
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Dynamic Programming

#38. Remove Nth Node From End of List #38 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Remove Nth Node From End of List.*

```

class Solution_38 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Remove Nth Node From End of List
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Graphs

#39. Copy List with Random Pointer #39 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Copy List with Random Pointer.*

```

class Solution_39 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Copy List with Random Pointer
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Dynamic Programming

#40. Add Two Numbers #40 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Add Two Numbers.

```
class Solution_40 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Add Two Numbers  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#41. Linked List Cycle #41 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Linked List Cycle.

```
class Solution_41 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Linked List Cycle  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#42. Find the Duplicate Number #42 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Find the Duplicate Number.

```
class Solution_42 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Find the Duplicate Number  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#43. LRU Cache #43 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for LRU Cache.

```
class Solution_43 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for LRU Cache  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#44. Merge K Sorted Lists #44 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Merge K Sorted Lists.

```
class Solution_44 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge K Sorted Lists
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#45. Reverse Nodes in k-Group #45 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Reverse Nodes in k-Group.

```
class Solution_45 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Nodes in k-Group
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#46. Invert Binary Tree #46 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Invert Binary Tree.

```
class Solution_46 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Invert Binary Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#47. Maximum Depth of Binary Tree #47 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Maximum Depth of Binary Tree.

```
class Solution_47 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Maximum Depth of Binary Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#48. Diameter of Binary Tree #48 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Diameter of Binary Tree.

```
class Solution_48 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Diameter of Binary Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#49. Balanced Binary Tree #49 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Balanced Binary Tree.

```
class Solution_49 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Balanced Binary Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#50. Same Tree #50 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Same Tree.

```
class Solution_50 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Same Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#51. Subtree of Another Tree #51 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Subtree of Another Tree.

```
class Solution_51 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Subtree of Another Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#52. Lowest Common Ancestor of a BST #52 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Lowest Common Ancestor of a BST.

```
class Solution_52 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Lowest Common Ancestor of a BST
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#53. Binary Tree Level Order Traversal #53 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Binary Tree Level Order Traversal.

```
class Solution_53 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Level Order Traversal
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#54. Binary Tree Right Side View #54 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Binary Tree Right Side View.

```
class Solution_54 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Right Side View
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#55. Count Good Nodes in Binary Tree #55 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Count Good Nodes in Binary Tree.

```
class Solution_55 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Count Good Nodes in Binary Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#56. Validate Binary Search Tree #56 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Validate Binary Search Tree.

```
class Solution_56 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Validate Binary Search Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#57. Kth Smallest Element in a BST #57 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Kth Smallest Element in a BST.

```
class Solution_57 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Kth Smallest Element in a BST
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#58. Construct Binary Tree from Preorder and Inorder Traversal #58 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Construct Binary Tree from Preorder and Inorder Traversal.

```
class Solution_58 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Construct Binary Tree from Preorder and Inorder Traversal
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#59. Binary Tree Maximum Path Sum #59 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Binary Tree Maximum Path Sum.*

```

class Solution_59 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Maximum Path Sum
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Dynamic Programming

#60. Serialize and Deserialize Binary Tree #60 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Serialize and Deserialize Binary Tree.*

```

class Solution_60 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Serialize and Deserialize Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Graphs

#61. Implement Trie (Prefix Tree) #61 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Implement Trie (Prefix Tree).*

```

class Solution_61 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Implement Trie (Prefix Tree)
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Dynamic Programming

#62. Design Add and Search Words Data Structure #62 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Design Add and Search Words Data Structure.*

```

class Solution_62 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Design Add and Search Words Data Structure
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

Graphs

#63. Word Search II #63 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Word Search II.

```
class Solution_63 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Search II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#64. Kth Largest Element in a Stream #64 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Kth Largest Element in a Stream.

```
class Solution_64 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Kth Largest Element in a Stream
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#65. Last Stone Weight #65 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Last Stone Weight.

```
class Solution_65 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Last Stone Weight
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#66. K Closest Points to Origin #66 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for K Closest Points to Origin.

```
class Solution_66 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for K Closest Points to Origin
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#67. Kth Largest Element in an Array #67 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Kth Largest Element in an Array.

```
class Solution_67 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Kth Largest Element in an Array
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#68. Task Scheduler #68 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Task Scheduler.

```
class Solution_68 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Task Scheduler
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#69. Design Twitter #69 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Design Twitter.

```
class Solution_69 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Design Twitter
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#70. Find Median from Data Stream #70 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Find Median from Data Stream.

```
class Solution_70 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Find Median from Data Stream
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#71. Subsets #71 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Subsets.

```
class Solution_71 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Subsets
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#72. Combination Sum #72 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Combination Sum.

```
class Solution_72 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Combination Sum
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#73. Permutations #73 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Permutations.

```
class Solution_73 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Permutations
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#74. Subsets II #74 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Subsets II.

```
class Solution_74 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Subsets II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#75. Combination Sum II #75 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Combination Sum II.

```
class Solution_75 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Combination Sum II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#76. Word Search #76 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Word Search.

```
class Solution_76 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Search
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#77. Palindrome Partitioning #77 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Palindrome Partitioning.

```
class Solution_77 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Palindrome Partitioning
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#78. Letter Combinations of a Phone Number #78 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Letter Combinations of a Phone Number.

```
class Solution_78 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Letter Combinations of a Phone Number
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#79. N-Queens #79 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for N-Queens.

```
class Solution_79 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for N-Queens
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#80. Number of Islands #80 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Number of Islands.

```
class Solution_80 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Number of Islands
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#81. Max Area of Island #81 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Max Area of Island.

```
class Solution_81 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Max Area of Island
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#82. Clone Graph #82 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Clone Graph.

```
class Solution_82 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Clone Graph
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#83. Walls and Gates #83 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Walls and Gates.

```
class Solution_83 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Walls and Gates  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#84. Rotting Oranges #84 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Rotting Oranges.

```
class Solution_84 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Rotting Oranges  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#85. Pacific Atlantic Water Flow #85 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Pacific Atlantic Water Flow.

```
class Solution_85 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Pacific Atlantic Water Flow  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#86. Surrounded Regions #86 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Surrounded Regions.

```
class Solution_86 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Surrounded Regions  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#87. Course Schedule #87 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Course Schedule.

```
class Solution_87 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Course Schedule
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#88. Course Schedule II #88 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Course Schedule II.

```
class Solution_88 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Course Schedule II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#89. Graph Valid Tree #89 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Graph Valid Tree.

```
class Solution_89 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Graph Valid Tree
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#90. Number of Connected Components #90 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Number of Connected Components.

```
class Solution_90 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Number of Connected Components
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#91. Redundant Connection #91 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Redundant Connection.

```
class Solution_91 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Redundant Connection
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#92. Word Ladder #92 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Word Ladder.

```
class Solution_92 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Ladder
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#93. Reconstruct Itinerary #93 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Reconstruct Itinerary.

```
class Solution_93 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reconstruct Itinerary
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#94. Min Cost to Connect All Points #94 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Min Cost to Connect All Points.

```
class Solution_94 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Min Cost to Connect All Points
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#95. Swim in Rising Water #95 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Swim in Rising Water.

```
class Solution_95 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Swim in Rising Water
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#96. Alien Dictionary #96 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Alien Dictionary.

```
class Solution_96 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Alien Dictionary
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#97. Cheapest Flights Within K Stops #97 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Cheapest Flights Within K Stops.

```
class Solution_97 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Cheapest Flights Within K Stops
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#98. Network Delay Time #98 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Network Delay Time.

```
class Solution_98 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Network Delay Time
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#99. Climbing Stairs #99 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Climbing Stairs.

```
class Solution_99 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Climbing Stairs
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#100. Min Cost Climbing Stairs #100 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Min Cost Climbing Stairs.

```
class Solution_100 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Min Cost Climbing Stairs
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#101. House Robber #101 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for House Robber.

```
class Solution_101 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for House Robber
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#102. House Robber II #102 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for House Robber II.

```
class Solution_102 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for House Robber II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#103. Longest Palindromic Substring #103 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Longest Palindromic Substring.

```
class Solution_103 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Longest Palindromic Substring
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#104. Palindromic Substrings #104 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Palindromic Substrings.

```
class Solution_104 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Palindromic Substrings
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#105. Decode Ways #105 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Decode Ways.

```
class Solution_105 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Decode Ways
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#106. Coin Change #106 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Coin Change.

```
class Solution_106 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Coin Change
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#107. Maximum Product Subarray #107 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Maximum Product Subarray.

```
class Solution_107 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Maximum Product Subarray
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#108. Word Break #108 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Word Break.

```
class Solution_108 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Break
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#109. Longest Increasing Subsequence #109 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Longest Increasing Subsequence.

```
class Solution_109 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Longest Increasing Subsequence
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#110. Partition Equal Subset Sum #110 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Partition Equal Subset Sum.

```
class Solution_110 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Partition Equal Subset Sum
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#111. Unique Paths #111 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Unique Paths.

```
class Solution_111 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Unique Paths
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#112. Longest Common Subsequence #112 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Longest Common Subsequence.

```
class Solution_112 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Longest Common Subsequence
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#113. Stock with Cooldown #113 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Stock with Cooldown.

```
class Solution_113 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Stock with Cooldown
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#114. Coin Change II #114 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Coin Change II.

```
class Solution_114 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Coin Change II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#115. Target Sum #115 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Target Sum.

```
class Solution_115 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Target Sum
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#116. Interleaving String #116 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Interleaving String.

```
class Solution_116 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Interleaving String
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#117. Longest Increasing Path in a Matrix #117 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Longest Increasing Path in a Matrix.

```
class Solution_117 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Longest Increasing Path in a Matrix
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#118. Distinct Subsequences #118 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Distinct Subsequences.

```
class Solution_118 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Distinct Subsequences
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#119. Edit Distance #119 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Edit Distance.

```
class Solution_119 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Edit Distance
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#120. Burst Balloons #120 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Burst Balloons.

```
class Solution_120 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Burst Balloons
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#121. Regular Expression Matching #121 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Regular Expression Matching.

```
class Solution_121 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Regular Expression Matching
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#122. Maximum Subarray #122 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Maximum Subarray.

```
class Solution_122 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Maximum Subarray
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#123. Jump Game #123 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Jump Game.

```
class Solution_123 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Jump Game
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#124. Jump Game II #124 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Jump Game II.

```
class Solution_124 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Jump Game II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#125. Gas Station #125 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Gas Station.

```
class Solution_125 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Gas Station
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#126. Hand of Straights #126 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Hand of Straights.

```
class Solution_126 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Hand of Straights
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#127. Merge Triplets to Form Target Array #127 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Merge Triplets to Form Target Array.

```
class Solution_127 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge Triplets to Form Target Array
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#128. Partition Labels #128 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Partition Labels.

```
class Solution_128 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Partition Labels
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#129. Valid Parenthesis String #129 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Valid Parenthesis String.

```
class Solution_129 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Valid Parenthesis String
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#130. Insert Interval #130 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Insert Interval.

```
class Solution_130 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Insert Interval
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#131. Merge Intervals #131 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Merge Intervals.

```
class Solution_131 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge Intervals
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#132. Non-Overlapping Intervals #132 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Non-Overlapping Intervals.

```
class Solution_132 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Non-Overlapping Intervals
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#133. Meeting Rooms #133 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Meeting Rooms.

```
class Solution_133 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Meeting Rooms
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#134. Meeting Rooms II #134 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Meeting Rooms II.

```
class Solution_134 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Meeting Rooms II
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#135. Minimum Interval to Include Each Query #135 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Minimum Interval to Include Each Query.

```
class Solution_135 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Minimum Interval to Include Each Query
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#136. Rotate Image #136 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Rotate Image.

```
class Solution_136 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Rotate Image
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#137. Spiral Matrix #137 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Spiral Matrix.

```
class Solution_137 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Spiral Matrix
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#138. Set Matrix Zeroes #138 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Set Matrix Zeroes.

```
class Solution_138 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Set Matrix Zeroes
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#139. Happy Number #139 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Happy Number.

```
class Solution_139 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Happy Number
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#140. Pow(x, n) #140 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Pow(x, n).

```
class Solution_140 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Pow(x, n)
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#141. Multiply Strings #141 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Multiply Strings.

```
class Solution_141 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Multiply Strings
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#142. Detect Squares #142 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Detect Squares.

```
class Solution_142 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Detect Squares
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#143. Single Number #143 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Single Number.

```
class Solution_143 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Single Number
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#144. Number of 1 Bits #144 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Number of 1 Bits.

```
class Solution_144 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Number of 1 Bits
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#145. Counting Bits #145 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Counting Bits.

```
class Solution_145 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Counting Bits
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#146. Reverse Bits #146 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Reverse Bits.

```
class Solution_146 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Bits
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#147. Missing Number #147 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Missing Number.

```
class Solution_147 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Missing Number
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#148. Sum of Two Integers #148 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Sum of Two Integers.

```
class Solution_148 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Sum of Two Integers
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Graphs

#149. Reverse Integer #149 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Reverse Integer.

```
class Solution_149 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Integer
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)

Dynamic Programming

#150. NeetCode Problem #150 #150 (Hard) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for NeetCode Problem #150.

```
class Solution_150 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for NeetCode Problem #150
        return input * 2
    }
}
```

Time Complexity: O(N)

Space Complexity: O(N)